10 - Stallo e Starvation

Concorrenza

La concorrenza si verifica quando più processi o thread vengono eseguiti contemporaneamente e devono condividere le stesse risorse. Questo principio è essenziale nei moderni sistemi operativi, in quanto consente di massimizzare l'efficienza e l'uso delle risorse disponibili. Tuttavia, la gestione della concorrenza richiede attenzione per evitare problemi di sincronizzazione e garantire un accesso corretto alle risorse condivise.

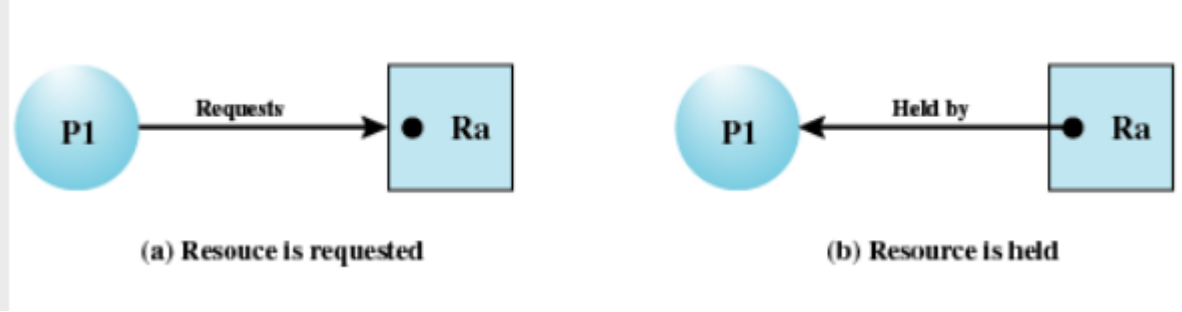
Stallo e Starvation

Definizione di Stallo (Deadlock)

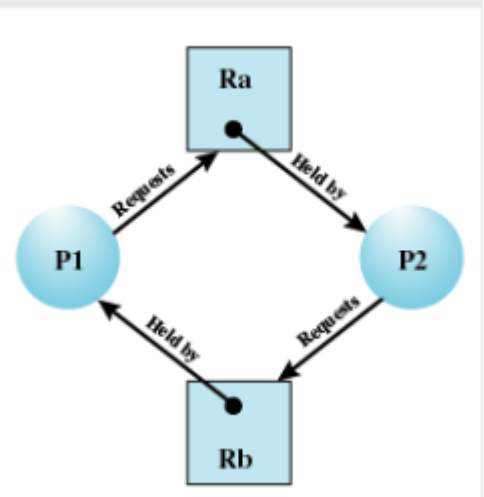
Lo stallo si verifica quando un insieme di processi rimane bloccato perché ciascuno di essi è in attesa di un'azione che può essere compiuta solo da un altro processo nello stesso insieme. Questo evento spesso riguarda il rilascio di risorse e, una volta che si verifica, nessuno dei processi coinvolti può proseguire l'esecuzione, rilasciare risorse o essere riattivato.

Grafici di Allocazione delle Risorse

Per comprendere meglio lo stallo, si possono usare **grafici di allocazione delle risorse**. Ad esempio, se un processo P1 richiede la risorsa Ra e questa è posseduta da un altro processo, si crea una situazione di dipendenza che potrebbe portare a un blocco, poiché Ra è posseduta già da P1.



Condizioni per lo Stallo



Affinché si verifichi uno stallo, devono essere presenti quattro condizioni:

- **Mutua esclusione**: Ogni risorsa può essere utilizzata da un solo processo alla volta.
- **Hold & Wait**: Un processo può mantenere le risorse già allocate mentre attende altre risorse.
- **Assenza di prerilascio**: Una risorsa non può essere forzatamente rilasciata da un processo.
- **Attesa circolare**: Esiste una catena di processi in cui ciascun processo attende una risorsa detenuta dal successivo nella catena.

Sebbene le prime tre condizioni derivino dalla progettazione del sistema e siano spesso inevitabili, l'attesa circolare può essere evitata attraverso strategie specifiche.

Strategie per affrontare il problema dello stallo

Esistono diversi approcci per gestire il problema dello stallo:

- **Ignorarlo** (ALGORITMO STRUZZO): Molti sistemi operativi, come Unix e Windows, accettano il rischio di stallo, preferendo un riavvio in caso di blocco.
- **Prevenirlo**: Si può progettare il sistema in modo da escludere una delle quattro condizioni necessarie allo stallo.
- **Evitare lo stallo**: Un algoritmo dinamico verifica ogni richiesta di risorse per assicurarsi che non conduca a una situazione di stallo.
- **Rilevarlo e risolverlo**: Si consente il verificarsi dello stallo, ma si adottano misure per identificarlo e risolverlo in seguito.

Prevenzione dello Stallo

Per prevenire lo stallo, si possono adottare due approcci:

- **Metodi indiretti**, che eliminano una delle tre condizioni necessarie.
- **Metodi diretti**, che evitano l'attesa circolare imponendo un ordine nelle richieste di risorse. Se un processo richiede una risorsa R, può richiedere successivamente solo una risorsa che nell'ordinamento segue R.

Un esempio di prevenzione è la negazione della mutua esclusione, che permette l'accesso simultaneo alle risorse condivisibili. Tuttavia, ciò non è sempre applicabile.

Evitare lo Stallo

Piuttosto che prevenirlo del tutto, alcuni sistemi operativi adottano strategie per evitare lo stallo:

- Un processo non viene avviato se la sua richiesta di risorse potrebbe portare a uno stallo.
- Una richiesta di risorse viene negata se potrebbe portare a un blocco nel sistema.

Un approccio comune è l'uso di un grafo di attesa, in cui la presenza di un ciclo indica un possibile stallo. Un altro metodo è il monitoraggio delle risorse disponibili rispetto a quelle richieste.

RIFIUTO DI ALLOCAZIONE DELLE RISORSE

Lo stato del sistema, in un sistema operativo, si riferisce a come sono distribuite le risorse tra i processi in un dato momento. Per tenere traccia di ciò, vengono utilizzate **matrici** che descrivono le **risorse** e la loro distribuzione. Le principali matrici sono:

1. **Risorse**: Un elenco di tutte le risorse disponibili nel sistema (ad esempio, CPU, memoria, stampanti, etc.).
2. **Disponibili**: Indica quante risorse di ciascun tipo sono libere e possono essere assegnate a un processo.
3. **Richieste**: Mostra quante risorse ogni processo sta richiedendo per completare la sua esecuzione. Queste sono risorse che un processo non ha ancora ricevuto, ma che richiede.
4. **Assegnazioni**: Indica quante risorse sono attualmente assegnate a ciascun processo.

In sostanza, queste matrici vengono utilizzate per avere una visione chiara della situazione attuale delle risorse nel sistema.

Quando un sistema assegna risorse a più processi, la domanda chiave è se il sistema possa continuare a funzionare correttamente o se rischia di entrare in uno **stallo** (deadlock), dove alcuni processi non riusciranno mai a proseguire la loro esecuzione a causa di risorse che non vengono mai rilasciate.

- **Stato sicuro**: Il sistema è in uno stato sicuro se esiste almeno una sequenza di esecuzione dei processi che permette di completare l'esecuzione di tutti i processi senza incorrere in un stallo. In altre parole, c'è un ordine in cui i processi possono ottenere le risorse, eseguire e terminare senza che nessuno di essi resti bloccato in attesa di risorse. Lo stato sicuro non fa avvenire deadlocks, diversamente dallo stato non sicuro. L'**avoidance** è l'assicurarsi che il sistema non entri in uno stato non sicuro

- **Stato non sicuro**: Il sistema è in uno stato non sicuro se non esiste una sequenza di esecuzione che permetta di terminare tutti i processi senza incorrere in un blocco. In questo caso, **potrebbe** verificarsi uno **stallo** in cui alcuni processi restano in attesa di risorse che non vengono mai rilasciate.

La strategia di esclusione dello stallo non si basa sulla certezza che uno stato sia di stallo, ma sulla **possibilità** che uno stato non sicuro determini uno stallo.

Quando un processo richiede una risorsa disponibile, il sistema :

- **Simula** l'assegnazione delle risorse e l'aggiornamento dello stato
- **Verifica** che il nuovo stato sia uno stato sicuro
- In caso **positivo** assegna le risorse,
- In caso **negativo** rifiuta la richiesta e il processo viene bloccato fino a che la risorsa non si rende disponibile

VANTAGGI NELL'EVITARE LO STALLO

Vantaggi: Non è necessario interrompere processi e riportarli in uno stato precedente (problema presente nelle strategie di rilevamento dello stallo)

Restrizioni:
• Il numero massimo di risorse necessitate da ogni processo deve essere noto a priori (prima dell'esecuzione)
• I processi devono essere indipendenti: l'ordine di esecuzione non deve essere vincolato a esigenze di sincronizzazione
• Deve esserci un numero fissato di risorse da allocare
• Quando un processo richiede risorse potrebbe essere posto in attesa
• Quando un processo ottiene tutte le risorse di cui necessita, deve rilasciarle in un tempo finito.

Rilevamento dello Stallo

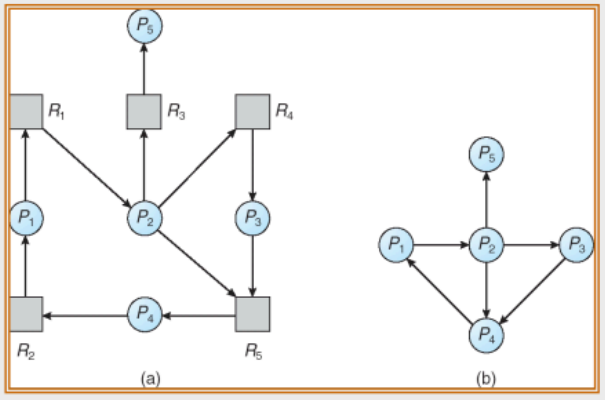
Se lo stallo non viene prevenuto o evitato, deve essere rilevato. Il rilevamento dello stallo avviene attraverso algoritmi che analizzano le allocazioni di risorse e individuano cicli nel

grafo di attesa.

Deadlock Detection: Rilevamento dello stallo

1. Quando è possibile le richieste vengono sempre soddisfatte
2. Il SO verifica periodicamente che non si sia verificato uno stallo
3. Il SO applica uno schema di recovery per risolvere lo stallo

Individuare lo stallo con una sola risorsa di ogni tipo



Resource-Allocation Graph *Corresponding wait-for graph*

- Un ciclo nel grafo di attesa denota uno stallo
- L'algoritmo può essere invocato periodicamente
- Numero di operazioni pari a n^2 dove n=numero di processi

16

Individuare lo stallo con più istanze per ogni risorsa

Consideriamo le matrici:
• **Available**: vettore di lunghezza m: ogni elemento indica il numero di istanze disponibili per ciascuna risorsa in un dato istante
• **Allocation**: matrice nxm: ogni elemento indica il numero di istanze di ciascuna risorsa assegnate a ciascun processo
• **Request**: matrice nxm: ogni elemento indica la richiesta di risorse da parte dei processi.

L'algoritmo di rilevamento indaga su ogni possibile sequenza di assegnazione per i processi che devono ancora essere completati. Vengono marcati i processi che non sono in stallo. Inizialmente nessun processo è marcato.

1. Si marca ogni processo che ha una riga di 0 nella matrice Allocation (il processo non ha risorse assegnate);
2. Si inizializza Work=Available;
3. Si cerca un indice i (processo) tale che:
    1. i non'è marcato (può essere in stallo);
    2. Request[i,:]≤Work; Se tale processo non esiste l'algoritmo termina
4. Se si trova il processo al punto 3. esso viene marcato e Work=Work+Allocation[i,:]

Un processo è in stallo solo se alla fine dell'algoritmo esso non risulta marcato.
L'algoritmo richiede un numero di operazioni dell'ordine di $O(mxn2)$

FILOSOFIA DELL'ALGORITMO:

- Trovare un processo le cui richieste possono essere soddisfatte con le disponibilità attuali
- Supporre che il processo vada a conclusione e rilasci le risorse
- Cercare un altro processo da soddisfare

	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Request matrix **Q**

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Allocation matrix **A**

R1	R2	R3	R4	R5
2	1	1	2	1

Resource vector

R1	R2	R3	R4	R5
0	0	0	0	1

Available vector

Figure 6.10 Example for Deadlock Detection

Utilizzo dell'algoritmo di rivelamento dello stallo

Quando e quanto spesso invocare l'algoritmo dipende da:

- Frequenza presunta con la quale si verifica lo stallo
- Numero di processi coinvolti nello stallo

In generale uno stallo si verifica quando un processo avanza una richiesta che non può essere soddisfatta immediatamente.
UTILIZZO DELL'ALGORITMO AD OGNI RICHIESTA. Consente la determinazione dello stallo e del processo la cui richiesta ha cagionato lo stallo. Questo comporta ad un aumento notevole del carico… (overhead)

INVOCARE L'ALGORITMO QUANDO LA PERCENTUALE DI UTILIZZO DELLE RISORSE SCENDE AL DISOTTO DI UNA SOGLIA

INVOCARE L'ALGORITMO AD ISTANTI **ARBITRARI**: nel grafo di assegnazione delle risorse potrebbero esistere molti cicli e diventa difficile determinare quale processo ha "causato" lo stallo

RIPRISTINO

Una volta individuato uno stallo, si può intervenire in due modi:

- **Terminazione dei processi coinvolti**: Si possono terminare tutti i processi bloccati o rimuoverli uno alla volta fino alla risoluzione del problema. Soluzione più comunemente adottata.

Abort di un processo alla volta fino a quando il ciclo non’è eliminato. Dopo ogni abort si deve rieseguire l’algoritmo di determinazione dello stallo. l'Ordine con cui abortire i processi è dato da funzione di minimo costo basata su:
– Priorità dei processi
– Tempo trascorso in esecuzione e necessario al completamento
– Risorse già utilizzate e risorse richieste (processo in fase di stampa…)
– Tipo di processo (interattivo, ecc.)

- **Prelazione delle risorse**: Si seleziona un processo e gli si sottraggono le risorse per sbloccare gli altri processi. Si attua un **rollback** del processo a cui è stata sottratta la risorsa, in modo da riavviarlo in seguito. In generale si preferisce **uccidere** il processo.
    - In oltre potrebbe avvenire una Starvation, ovvero quando un processo o più vengono selezionati costantemente come vittime, quindi i processi con **bassa priorità** o che consumano risorse critiche, vengano ripetutamente interrotti e mai portati a termine. Questo succede perché, per evitare il deadlock, il sistema potrebbe preferire sempre "uccidere" il processo che è in attesa di risorse, ma che non sta avanzando, senza mai permettergli di completare la sua esecuzione.

Un approccio per bilanciare il rischio di starvation è quello di **includere il numero di rollback** nel calcolo del costo. Un **rollback** avviene quando un processo viene terminato o interrotto, e tutte le operazioni fatte fino a quel punto vengono annullate (cioè, le risorse allocate e le modifiche fatte dal processo vengono "riportate indietro"). Ogni volta che un processo viene fatto **rollback**, il sistema deve fare un lavoro extra per recuperare lo stato precedente.

Se un processo viene interrotto o fatto rollback troppe volte, potrebbe essere più costoso per il sistema continuare a forzare il rilascio delle risorse da parte di quel processo. Pertanto, il sistema può usare il **numero di rollback** come un **fattore di costo** per decidere se un processo deve essere scelto come vittima o meno.

Starvation

Un altro problema legato alla concorrenza è la starvation, che si verifica quando alcuni processi rimangono perennemente in attesa perché le risorse vengono sempre assegnate a processi con priorità più alta. Questo problema è tipico dei sistemi di scheduling basati sulla priorità.

Una soluzione comune alla starvation è l'**aging**, un meccanismo che aumenta gradualmente la priorità dei processi in attesa per garantire che prima o poi vengano eseguiti.